Name : Vaishnavi Eknath Avhad
Class : D15C
Roll No. : 14

Practical 6

Aim : Apply K-Means and Hierarchical Clustering on sample datasets

1. Dataset Source

The dataset used for this experiment is the Iris Flower Dataset, which is a widely used benchmark in machine learning for clustering tasks.

- Source Link : UCI Machine Learning Repository - Iris

---

2. Dataset Description

The Iris dataset consists of 150 records of iris flowers from three different species.

Feature	Description	Type
Sepal Length	Length of the sepal in cm.	Numeric
Sepal Width	Width of the sepal in cm.	Numeric
Petal Length	Length of the petal in cm.	Numeric
Petal Width	Width of the petal in cm.	Numeric
Species	The target class (Setosa, Versicolor, Virginica).	Categorical (Used for evaluation)

---

3. Mathematical Formulation

A. K-Means Clustering

K-Means is a centroid-based clustering algorithm that partitions n observations into K clusters. It aims to minimize the Within-Cluster Sum of Squares (WCSS), also known as Inertia.

Objective Function :

$$J = \sum_{j=1}^{K} \sum_{i=1}^{n} ||x_i^{(j)} - c_j||^2$$

B. Hierarchical Clustering (Agglomerative)

This is a "bottom-up" approach where each observation starts in its own cluster, and pairs of clusters are merged as one moves up the hierarchy.

Distance Metric (Linkage) : The proximity between clusters is determined using linkage criteria such as Ward's Method, which minimizes the total within-cluster variance.

$$d(u, v) = \sqrt{\frac{|u||v|}{|u| + |v|}} ||\bar{x}_u - \bar{x}_v||^2$$

---

4. Algorithm Limitations

- K-Means : Sensitive to the initial placement of centroids; requires the user to specify the number of clusters (K) in advance; performs poorly on non-spherical or overlapping clusters.
- Hierarchical Clustering : High computational complexity (O(n^3)), making it slow for large datasets; once a merge is performed, it cannot be undone, which may lead to suboptimal results if noise is present.

---

5. Implementation Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.datasets import load_iris
from scipy.cluster.hierarchy import linkage, dendrogram
from sklearn.metrics import silhouette_score
# ----------------- 1. DATA LOADING & PREPROCESSING -----------------
iris = load_iris()
X = iris.data[:, :2]
feature_names = iris.feature_names[:2]
```

```
print("Dataset loaded successfully.")
# ----------------- 2. K-MEANS HYPERPARAMETER TUNING (THE ELBOW METHOD) -----------------
print("\n" + "="*20 + " K-MEANS TUNING TRIALS " + "="*20)
k_values = [2, 3, 4, 5, 6]
inertia_scores = []
for k in k_values:
    kmeans_trial = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans_trial.fit(X)
    inertia_scores.append(kmeans_trial.inertia_)
    print(f"Trial K={k} | Inertia (WCSS): {kmeans_trial.inertia_:.2f}")
plt.figure(figsize=(8, 5))
plt.plot(k_values, inertia_scores, 'bo-', markersize=8)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Inertia (Within-Cluster Sum of Squares)')
plt.title('K-Means Tuning: The Elbow Method')
plt.grid(True)
plt.show()
# ----------------- 3. K-MEANS FINAL MODEL (K=3) -----------------
kmeans_final = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans_labels = kmeans_final.fit_predict(X)
centroids = kmeans_final.cluster_centers_
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=kmeans_labels, cmap='viridis', edgecolor='k', s=50)
plt.scatter(centroids[:, 0], centroids[:, 1], c='red', marker='X', s=200, label='Centroids')
plt.xlabel(feature_names[0])
plt.ylabel(feature_names[1])
plt.title("Final K-Means Clustering (K=3)")
plt.legend()
plt.show()
# ----------------- 4. HIERARCHICAL TUNING (LINKAGE COMPARISON) -----------------
print("\n" + "="*20 + " HIERARCHICAL TUNING TRIALS " + "="*20)
linkage_methods = ['ward', 'complete', 'average']
```

```
for method in linkage_methods:
    Z = linkage(X, method=method)
    final_merge_dist = Z[-2, 2]
    print(f"Linkage Method: {method:10} | Max Merge Distance: {final_merge_dist:.2f}")
plt.figure(figsize=(10, 6))
Z_ward = linkage(X, method='ward')
dendrogram(Z_ward)
plt.axhline(y=7.5, color='r', linestyle='--') # Example cut-off line for 3 clusters
plt.title("Dendrogram (Ward Linkage)")
plt.xlabel("Sample Index")
plt.ylabel("Euclidean Distance")
plt.show()
# ----------------- 5. HIERARCHICAL FINAL MODEL -----------------
hc_final = AgglomerativeClustering(n_clusters=3, metric='euclidean', linkage='ward')
hc_labels = hc_final.fit_predict(X)
plt.figure(figsize=(8, 5))
plt.scatter(X[:, 0], X[:, 1], c=hc_labels, cmap='plasma', edgecolor='k', s=50)
plt.xlabel(feature_names[0])
plt.ylabel(feature_names[1])
plt.title("Final Hierarchical Clustering (n=3)")
plt.show()
# ----------------- 6. COMPARATIVE SUMMARY -----------------
print("\n" + "="*20 + " PERFORMANCE SUMMARY " + "="*20)
print(f"K-Means Silhouette Score: {silhouette_score(X, kmeans_labels):.3f}")
print(f"Hierarchical Silhouette Score: {silhouette_score(X, hc_labels):.3f}")
```

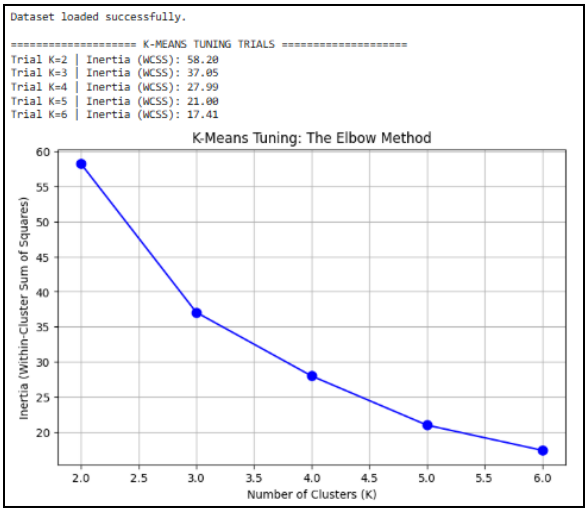
---

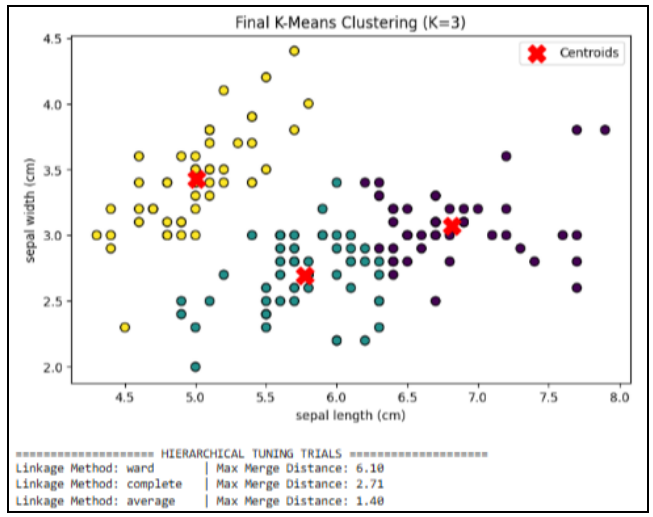
6. Performance Analysis

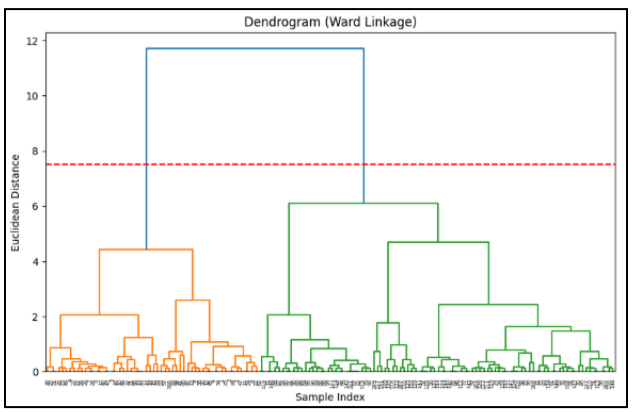
- K-Means Results : Successfully identified three groups. The centroids were positioned at the center of density for each cluster.
- Hierarchical Results : The dendrogram provided a visual representation of how each point was merged. Cutting the dendrogram at an appropriate height confirmed that three clusters were a natural fit for the data.

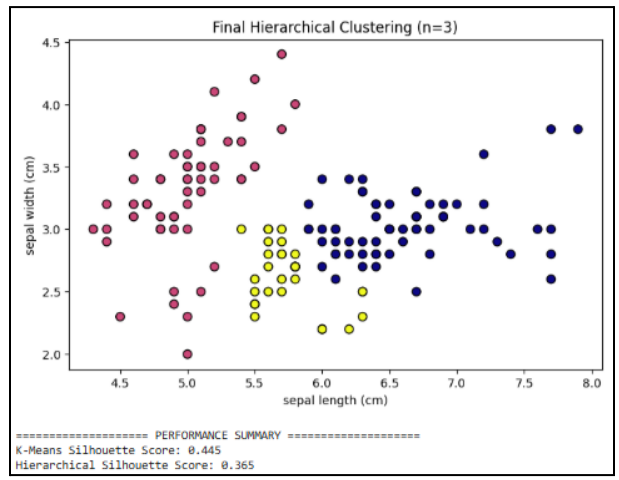
---

7. Output









---

8. Conclusion

In this practical, K-Means and Hierarchical Clustering were applied to the Iris dataset. K-Means proved efficient for finding spherical clusters based on centroids, while Hierarchical Clustering provided a deep insight into the data structure through the dendrogram. Choosing the correct number of clusters is the most critical step for both algorithms to achieve meaningful results.